AI110 | Foundations of AI Engineering

Foundations of AI Engineering Spring 2026 (@ Observer access)
Personal Member ID#: **133118**

Welcome

Getting Started

▶ Course Info

AI Prompting Guide

Week 1

Week 2

Week 3

Week 4

Week 5

Week 6

Week 7

Week 8

Week 9

Week 10

▶ Project Grading

Instructor | Overview | Tinker | Show | Report

Tinker: The Mood Machine

ℹ️ Activity Overview

⏰ **~45 minutes**

You've been handed an early prototype of The Mood Machine, a tiny sentiment analyzer meant to scan short posts and guess whether they sound positive, negative, neutral, or mixed. The twist: this system is not "smart." It has no idea what emotions are. It starts with a handful of keywords and a rigid set of rules that you must refine.

In this lab, you'll tinker with two different approaches to AI:

1. **A rule based classifier** that follows the exact logic you write.
2. **A simple machine learning classifier** that learns patterns from labeled examples you create.

Your mission is to explore how both systems behave, break them, fix them, measure them, and then document their strengths and weaknesses. Along the way, you'll uncover why seemingly simple tasks in AI quickly become messy, brittle, and full of hidden assumptions.

🎯 Goals

By completing this activity, you will be able to...

- [ ] Modify a rule based classifier and understand how handcrafted logic shapes predictions.
- [ ] Build and label a small dataset and see how data choices influence both rule based and ML systems.
- [ ] Identify where simple models fail, including sarcasm, slang, emojis, ambiguity, and cultural context.
- [ ] Train and evaluate a tiny ML model to compare learned behavior with manual rules.
- [ ] Write a model card that clearly communicates how your system works and where it breaks.

✏️ Tinker Instructions

▾ Part 1: Build the Dataset

⏰ **~5 mins**

Every AI system begins with data. Your Mood Machine is no different: it only knows the words you give it and the labels you assign. Right now, its "worldview" is tiny. It sees a handful of example posts and a short list of positive and negative words.

In this step, you'll explore and expand the dataset that drives both versions of the Mood Machine: the rule based classifier and the optional ML classifier. By adding new posts and labels, you'll directly shape what the model can and cannot understand.

- [ ] Go to the **The Mood Machine** starer repo and fork it to your GitHub account.
- [ ] Clone your fork and open the folder in VS Code.
- [ ] Open `dataset.py`.Take a moment to understand how this file is structured.

  Use Copilot to help you inspect the dataset.

  - Reference the file or a selection when asking questions.
  - Focus on understanding relationships between `SAMPLE_POSTS` and `TRUE_LABELS`.
  - If Copilot's explanation feels generic, add more context or reason through it yourself.

- [ ] Read through the existing posts and labels. Discuss with your group:
  - Which posts are easy to label?
  - Which ones feel ambiguous or mixed?
  - Would everyone agree on these labels?
- [ ] Add **5-10** new posts to `SAMPLE_POSTS` that reflect realistic language: slang, emojis, mixed emotions, or subtle tone.

  If you want ideas, treat Copilot as a brainstorming partner. Keep this a short, task-specific chat, and evaluate whether its suggestions sound like real human language. Reject anything that feels artificial, repetitive, or unclear.

- [ ] Add matching labels to `TRUE_LABELS` for each new post.

  Every time you add a post, you must add **one label**. A quick check:

  ```
  len(SAMPLE_POSTS) == len(TRUE_LABELS)
  ```

  If these lengths don't match, the program will crash later.

- [ ] Run the rule based version of the project:

```
python main.py
```

You should see it evaluate your posts and show predictions. If you hit an error, use Copilot to interpret the error *in the context of this project*. Verify the explanation by checking the file and list lengths yourself.

📍 **Checkpoint**: The program runs without errors, and your expanded dataset loads correctly.

Close Section

▾ Part 2: Engineer the Rule-Based Brain

⏰ **~10 mins**

Your Mood Machine now has data, but no reasoning. To classify text, it needs a small engine that reads a sentence, checks for mood signals, and converts that information into a score.

Inside `mood_analyzer.py`, you'll construct this "brain." Every choice you make shapes how your system interprets language — and exposes where it fails. This kind of rule based reasoning is a stepping stone to understanding why machine learning models eventually became necessary.

- [ ] Open the file `mood_analyzer.py`. You'll see three core methods `preprocess`, `score_text`, and `predict_label`. These work together. Take a moment to understand how data flows through them.

  If the structure feels unfamiliar, briefly use Copilot to reason about the file as it exists.

- [ ] Start with `preprocess`. Improve it so the model works with consistent text. For example, lowercasing and simple tokenization are good starting points. Print the tokens to confirm your code behaves as expected.

  If you're unsure what to change, use Copilot to plan improvements before editing. Ask it to reason about tradeoffs, and decide which changes actually matter for this dataset. After making changes, print intermediate results to verify behavior.

- [ ] Move on to `score_text`. This is where you design how the model interprets words. Your model should:
  - Loop over tokens
  - Increase or decrease a score based on matches
  - Return the total score

  Begin with a simple approach, then choose one intentional enhancement (for example: negation handling, word weighting, or emoji signals). Use Copilot in **Edit** or **Plan** mode to refactor messy logic, explore alternative implementations, and compare readability vs correctness.

  Think of scoring like this:
  - Start at 0
  - Positive signal → add points
  - Negative signal → subtract points

  The score is not magic. It's just a number that summarizes what the model noticed.

- [ ] In `predict_label`, map your score to a label like **positive**, **negative**, **neutral**, or **mixed**. Adjust thresholds if needed based on how your scoring behaves.

  A simple starting rule:
  - Score > 0 → positive
  - Score < 0 → negative
  - Score == 0 → neutral

  If your model feels too sensitive, you can require stronger scores before assigning a label.

- [ ] Run your program:

```
python main.py
```

- [ ] Test a few obvious cases:
  - A clearly positive sentence
  - A clearly negative sentence
  - A neutral or flat statement

- [ ] If predictions seem off, print intermediate values (tokens, scores, matches). Use Copilot to help explain what your current logic is doing, and inspect tokens, scores, and conditionals yourself.

  Avoid long chat threads. If your goal changes from "understand" to "modify," start a fresh session.

📍 **Checkpoint**: Your rule-based classifier produces reasonable predictions for simple cases, and you can explain why it made each decision based on your logic.

Close Section

▾ Part 3: Stress Test and Break the Model

⏰ **~10 mins**

Your Mood Machine now works — but that doesn't mean it works well. Rule-based systems are brittle by nature. They struggle with context, sarcasm, slang, and anything that doesn’t fit cleanly into predefined rules.

In this step, you'll stress test your system to uncover those weaknesses. You'll use GitHub Copilot as an analysis partner to help explain failures and reason about possible fixes, while staying fully responsible for deciding what to change and why.

- [ ] Run your current rule-based system:

```
python main.py
```

- [ ] Create a small set of "breaker" sentences designed to confuse your model. Focus on realistic language, such as:
  - Sarcasm ("I love getting stuck in traffic")
  - Slang with multiple meanings ("sick," "wicked," "fire")
  - Emojis ("I'm fine 🙂")
  - Mixed emotions ("I'm exhausted but proud of myself")

- [ ] Run each breaker through your Mood Machine and observe the output. Pay attention to:
  - Which words affected the score
  - Which words were ignored
  - Whether one token dominated the decision
- [ ] Identify one clear failure pattern. For example:
  - Sarcasm interpreted as positive due to a single keyword
  - Slang misclassified because it isn't in the word lists
  - Negation being ignored
  - Emojis having no effect
- [ ] Pick **one failure** that you want to fix. Open either `dataset.py` (to update your vocabulary), or `mood_analyzer.py` (to improve your scoring logic).

  Use Copilot to reason about why the failure is happening. If it proposes a fix, treat it as a hypothesis, not an answer.

- [ ] Make a targeted change to address the failure. Keep the change small and intentional.
- [ ] Re-run the same breaker. Did your change help? Did it accidentally break other examples?

  If the label is still not perfect but *less wrong* than before, that's progress. Real models improve gradually, not all at once.

📍 **Checkpoint**: You can show at least one sentence your model originally misclassified, explain why it failed, and demonstrate how a targeted change altered its behavior — even if the fix introduced new limitations.

Close Section

▾ Part 4: Evaluate Your System

⏰ **~10 mins**

Your Mood Machine runs. You've added rules. You've patched failures. Now comes a critical engineering step: evaluation.

In this part, you'll measure how your system behaves on labeled data and practice using Copilot as a verification and interpretation tool, not a judge. You'll learn why accuracy alone is never enough, and why engineers must inspect results even when the numbers look good.

- [ ] Open `dataset.py` and confirm that every post in `SAMPLE_POSTS` has a corresponding label in `TRUE_LABELS`. Add more examples if you haven't already.
- [ ] Run your rule-based model:

  ```
  python main.py
  ```

  At the top of the output, you'll see an automatic evaluation: each post, the predicted label, the true label, and a final accuracy score.
- [ ] Study the mismatches. Which examples did the model get wrong? What patterns do you notice in the failures?
- [ ] Choose **one incorrect prediction** and analyze it in detail. Use Copilot to help explain what your current logic is doing. Ask for a reasoning walkthrough, not a fix. Compare Copilot's explanation to your understanding.
- [ ] Decide whether the error is something you want to fix now or document as a limitation.

  > Not all errors are worth fixing. Some reveal fundamental limits of rule-based systems. Choosing *not* to change the model is sometimes the correct engineering decision.

- [ ] Try the ML version:

  ```
  python ml_experiments.py
  ```

  This trains a tiny classifier on your labeled posts. Compare its accuracy and individual predictions to the rule based version. Does it fail in the same places, or in different ones?

  > If the ML results surprise you, use Copilot to reason about why the model might behave differently given the same dataset. Treat any explanation as a hypothesis and validate it against the code and data.

- [ ] Add a few more labeled posts to see how changing the dataset affects each model. Which approach seems more sensitive to the data you created?

📍 **Checkpoint**: You have completed this step when you can describe:

- which examples your model gets right or wrong.
- why some failures happen, and
- how rule-based and ML systems behave differently on the same dataset.

Close Section

▾ Part 5: The Model Card

⏰ **~10 mins**

You've built a functioning Mood Machine, expanded its vocabulary, patched its weaknesses, and evaluated its behavior. Now you'll document it.

A model card is a short, structured way to explain:

- what your system is meant to do,
- how it works,
- what data it relies on,
- where it fails, and
- who it may or may not work well for.

Even simple systems need transparent documentation. This is part of responsible AI practice: a model should never be a mystery box.

- [ ] Open `model_card.md`. The template includes headings for intended use, data, how the model works, evaluation, limitations, and ethical considerations.
- [ ] Fill in the sections based on the system *you* built. Include details such as:
  - How you expanded `SAMPLE_POSTS` and how you labeled them
  - The scoring rules you wrote in `mood_analyzer.py`
  - Any enhancements you added (negation handling, slang, emoji logic, etc.)
  - The patterns you observed in your evaluation results
  - The sentences that consistently confused your model
- [ ] Describe the known limitations using examples from Part 3 and Part 4. Be specific. Instead of saying “struggles with sarcasm,” show an example the model misclassified and explain why.
- [ ] Add a short note on **bias and scope**. Think about the language you included in your dataset — slang, tone, dialect, or cultural references. Who is this model optimized for? Who might it misinterpret?

- [ ] Based on the ML extension in `ml_experiments.py`, add a short comparison:
  - Did the learned model behave differently from your rule based one?
  - Did it fix certain failures or introduce new ones?
  - How sensitive was it to the labels you created?

📍 **Checkpoint**: Your model card is complete when a reader could open your repo, read this document, and clearly understand:

- how your Mood Machine works,
- what data shaped it,
- where it succeeds,
- where it fails, and
- why these limitations matter.

Close Section